

0.1 Introduction

This chapter presents the Critical Path Toolkit, which implements the schedule and cost reading of the network model. In this reading the value attached to a node is a quantity that accumulates along dependency paths, such as a duration or a cost, and the value attached to an edge is the amount added or applied in the transfer between two components. The toolkit reads the graph object of Chapter 3 directly. Every analysis in it is a linear-time pass over the layered iteration sets, and the Network Decomposition Module of Chapter 4 is not used.

For a network carrying such a quantity, a forward pass gives the best or worst value that a complete chain of dependencies can produce. The margin of a component is how far it can move before that value changes. The margin identifies the critical components, and it can be computed only when the arithmetic of the quantity can be run backwards.

The contributions of this chapter are the following. The classical critical path method is generalised to a family of analysis modes, each defined by a pair of operators, and the condition under which a mode has a backward pass is stated and used to select the modes provided. Two propagation kernels are given that serve every mode. For interval durations, the forward quantities are computed exactly from two corner runs, and the floats are computed exactly by a domination split that reduces the enumeration from every uncertain duration to those that can bypass the node in question. Every result is checked against an independent oracle.

The rest of the chapter is structured as follows. Section 0.2 recalls the results from Chapter 2 on which the toolkit builds. Section 0.3 defines the analysis modes and Section 0.4 the two kernels. Section 0.5 describes the treatment of interval durations, and Section 0.6 presents the domination split and proves its exactness. Section 0.7 describes the validation of the toolkit. Section 0.8 applies it to a benchmark project network and locates the boundary of the split on a dense mesh. Section 0.9 discusses limitations and extensions, and Section 0.10 concludes the chapter.

0.2 Background

Chapter 2 reviewed the critical path method [1], its float calculus, and the treatment of duration uncertainty from PERT [2] to the interval results of Chanas, Zieliński and their co-authors. When each duration is an interval, the forward quantities are monotone in the durations, and their exact bounds are found at the two corners of the input box. The backward quantities are not monotone. Deciding whether an activity is possibly critical is NP-complete and computing float bounds is NP-hard [3–6]. The interval treatment of Sections 0.5 and 0.6 rests on these two facts.

The forward and backward passes of the classical method are linear algebra over

an idempotent semiring, and the backward pass is a residuation [7]. Hence a forward recursion exists for any pair of combination and compounding operations, but a backward pass exists only when the algebra of the pair supports one. A slack reported for a pair without such a backward pass is a number with no meaning. The modes of Section 0.3 rest on this fact.

0.3 Analysis Modes

The forward recursion of the critical path method combines the values arriving at a node from its parents and compounds the result with the node's own value. Writing $\oplus$ for the combination and $\otimes$ for the compounding, the classical method has $\oplus = \max$ and $\otimes = +$. Other pairs give other analyses: $\min$ and $+$ give the fastest chain, $\max$ and $\times$ give the best multiplicative chain, and $+$ and $+$ give a roll-up in which every route contributes. Whether a margin is defined is a property of the pair.

When $\oplus$ is an order ($\max$ or $\min$) and $\otimes$ is monotone with a residual, it is. A bound on the combined value then binds every argument, and the bound inverts across each edge, so a constraint of the form “finish by K ” travels backwards through the network in the same way that values travel forwards. This is residuation in an idempotent semiring [7], and it is what the classical backward pass is. The backward object is a margin: the distance between the best complete path and the best complete path forced through the node in question, expressed as a difference for the additive pairs and as a ratio for the multiplicative one. The zero set of the margin is the critical structure of the analysis.

When $\oplus$ is a sum, a total is not a deadline and no residuation exists. However, a summed system is linear, and a linear map has an adjoint. The backward object is therefore a sensitivity, and the margin it defines is an allowance against a stated budget. It is not a slack against a deadline. A pair with neither structure supports a forward recursion and nothing else, and no margin is defined for it.

A configuration of the toolkit is called a mode. A mode is an operator pair together with the backward semantics its algebra supports. Table 1 lists the four modes provided. The MaxScaling mode is applied in Chapter ?? . The three order-based modes share one kernel and one backward mechanism, described in Section 0.4.1. The accumulation mode has its own kernel, described in Section 0.4.2.

0.4 The Propagation Kernels

0.4.1 The order-based kernel

For the order-based modes one fold serves both directions. Let d_v be the value of node v , w_{uv} the value of edge (u, v) , $\text{pa}(v)$ the parents of v and $\text{succ}(v)$ its successors. Read

Table 1: The modes provided by the toolkit. In the three order-based modes the backward quantities are computed by the same fold on the reversed graph and the margin is the gap between the project value and the value through the node. In the accumulation mode the backward quantity is the adjoint, a path count.

Mode	$\oplus$	$\otimes$	Backward object	Margin
LongestPath	max	+	reverse fold, margin $P - t_v$	slack
ShortestPath	min	+	reverse fold, margin $t_v - P$	margin over optimum
MaxScaling	max	$\times$	reverse fold, margin $P/t_v - 1$	ratio slack
Accumulation	+	+	adjoint (path multiplicity)	allowance under a budget

forwards over the layered iteration sets, the fold computes

$$F_v = \left(\bigoplus_{u \in \text{pa}(v)} F_u \otimes w_{uv} \right) \otimes d_v, \quad (1)$$

the best value that a chain of dependencies can deliver into each node, with each source seeded from an initial value. Read over the reversed graph, the same fold computes

$$R_v = \bigoplus_{s \in \text{succ}(v)} (R_s \otimes d_s) \otimes w_{vs}, \quad (2)$$

the best completion from each node onwards, with each sink seeded at the neutral element of $\otimes$. The two arrays meet at each node. Their compound

$$t_v = F_v \otimes R_v \quad (3)$$

is the best complete path forced through v . The project value P is the combination of the forward values at the sinks, and the margin of v is the gap between the two,

$$m_v = P - t_v \text{ (LongestPath)}, \quad m_v = t_v - P \text{ (ShortestPath)}, \quad m_v = P/t_v - 1 \text{ (MaxScaling)}. \quad (4)$$

A node is critical when its margin is zero. For the additive modes the textbook schedule quantities are recovered from the same two arrays,

$$\text{ES}_v = F_v - d_v, \quad \text{LF}_v = P - R_v, \quad \text{LS}_v = \text{LF}_v - d_v. \quad (5)$$

Algorithm 1 gives the kernel.

On the four-node network of Figure 1, with durations 1, 5, 2, 1 and no edge delays, LongestPath completes in 7 along $1 \rightarrow 2 \rightarrow 4$ and node 3 carries slack 3. ShortestPath completes in 4 along $1 \rightarrow 3 \rightarrow 4$ and node 2 carries margin 3. The two modes select different chains from the same inputs.

Algorithm 1 Order-based kernel (one mode)

Require: layered iteration sets L , incoming and outgoing indices, sources S , node values d , edge values w , mode $(\oplus, \otimes)$ with neutral element e

Ensure: F , R , P , t , m , critical set

```
1: for all layers  $L_i$  in order, nodes  $v \in L_i$  do
2:   if  $v \in S$  then  $F_v \leftarrow e \otimes d_v$ 
3:   else  $F_v \leftarrow (\oplus_{u \in \text{pa}(v)} F_u \otimes w_{uv}) \otimes d_v$ 
4:   end if
5: end for
6: for all layers  $L_i$  in reverse order, nodes  $v \in L_i$  do
7:   if  $\text{succ}(v) = \emptyset$  then  $R_v \leftarrow e$ 
8:   else  $R_v \leftarrow \oplus_{s \in \text{succ}(v)} (R_s \otimes d_s) \otimes w_{vs}$ 
9:   end if
10: end for
11:  $P \leftarrow \oplus_{v \in \text{sinks}} F_v$ 
12: for all nodes  $v$  do
13:    $t_v \leftarrow F_v \otimes R_v$ ;    $m_v \leftarrow \text{margin}(P, t_v)$  by Eq. (4)
14: end for
15:  $\text{critical} \leftarrow \{v : m_v = 0\}$ 
```

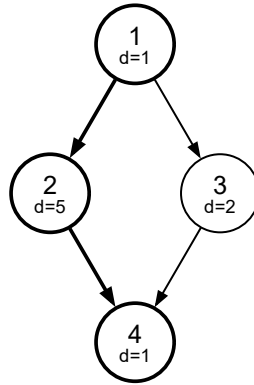


Figure 1: The running example. Node durations 1, 5, 2, 1. LongestPath is critical through nodes 1, 2, 4 and ShortestPath through 1, 3, 4.

0.4.2 The linear kernel

In the accumulation mode the combination is a sum, so

$$F_v = \sum_{u \in \text{pa}(v)} (F_u + w_{uv}) + d_v. \quad (6)$$

A node's value therefore reaches a downstream target once per directed route. The total at a target t expands to

$$F_t = \sum_v m_v d_v + \sum_{(u,v) \in E} m_v w_{uv}, \quad (7)$$

where m_v is the number of directed routes from v to t . The counts are computed by a reverse traversal seeded with $m_t = 1$,

$$m_v = \sum_{s \in \text{succ}(v)} m_s, \quad (8)$$

which is the adjoint of the forward map. The count m_v is the derivative of the total with respect to the node's value, and the product $m_v d_v$ is the node's contribution to the total. Under a budget B the allowance of node v is

$$a_v = \frac{B - F_t}{m_v}, \quad (9)$$

the headroom divided across the node's routes. Algorithm 2 gives the kernel. On the running example the total at node 4 is 10 and node 1 counts twice, once through each branch. If no target is given the toolkit takes the sink with the largest forward value.

Algorithm 2 Linear kernel (accumulation mode)

Require: layered iteration sets L , indices, sources S , node values d , edge values w , target t (default: sink with largest F), optional budget B

Ensure: F , total F_t , multiplicities m , contributions, allowances

```

1: for all layers  $L_i$  in order, nodes  $v \in L_i$  do
2:    $F_v \leftarrow d_v$  if  $v \in S$ , else  $\sum_{u \in \text{pa}(v)} (F_u + w_{uv}) + d_v$ 
3: end for
4:  $m_t \leftarrow 1$ 
5: for all layers  $L_i$  in reverse order, nodes  $v \in L_i, v \neq t$  do
6:    $m_v \leftarrow \sum_{s \in \text{succ}(v)} m_s$ 
7: end for
8: for all nodes  $v$  with  $m_v > 0$  do
9:    $\text{contribution}_v \leftarrow m_v d_v$ ; if  $B$  given,  $a_v \leftarrow (B - F_t)/m_v$ 
10: end for
```

0.4.3 Cost

Both kernels touch each node once and each edge a constant number of times, so every mode costs $O(|V| + |E|)$ whatever the operators are. The kernels are generic over the value type they propagate, and the concrete types are described in Chapter 8.

0.5 Interval Durations

When the durations are intervals, the quantities of the toolkit divide into two classes according to monotonicity.

Every forward quantity, F_v , R_v , t_v and P , is monotone in every input. Hence its exact bounds are obtained from two scalar runs of the kernel, one with every input at its lower endpoint and one with every input at its upper endpoint, and interval arithmetic is not needed. The toolkit computes the forward quantities in this way for every mode.

A margin is a difference of two monotone quantities that share their inputs, so it is not monotone in anything, and the hardness results of Section 0.2 apply to it. The toolkit provides three methods for the margins, and each result is tagged with the method that produced it, so that an exact range and an enclosure cannot be confused.

1. *Conservative enclosure.* From the exact bounds on P and t_v , the LongestPath margin lies within

$$\left[\max(0, \underline{P} - \bar{t}_v), \bar{P} - \underline{t}_v \right], \quad (10)$$

and correspondingly for the other modes. The enclosure is sound and costs no more than the two corner runs. It is not exact, because the lower corner of P and the upper corner of t_v are not in general attained by the same configuration of durations.

2. *Exhaustive corner enumeration.* The margin extremes are attained at corner configurations (Proposition 1 below), so enumerating every corner of the non-degenerate node and edge intervals and running the kernel at each gives the exact margin range. The cost is 2^k runs for k interval inputs, and the toolkit refuses beyond a stated cap unless the cap is raised.
3. *Domination split.* For the LongestPath mode with point-valued edge values, the exact margin range is obtained from a much smaller enumeration, described in Section 0.6.

From the margin bounds the toolkit reports the two criticality sets of the interval PERT literature [3]. A node is *necessarily critical* when the upper bound of its margin is zero, that is, it is critical in every configuration of the box. A node is *possibly critical* when the lower bound of its margin is zero, that is, it is critical in at least one configuration.

Under the conservative enclosure the necessarily critical set is certain and the possibly critical set is a superset that the exact methods tighten. Degenerate intervals, whose two endpoints coincide, are held fixed by every method and do not enter the enumeration.

An earlier implementation of the toolkit ran the scalar kernels on an interval type with overloaded arithmetic, so that $\max$, $+$ and $-$ were applied to intervals directly. It was withdrawn because it produced wrong results at the combination and margin steps. In the combination step, $\max$ of two overlapping intervals is not defined, since neither contains the other. Taking the endpoint-wise maximum, which is what the overloaded operator did, returns an interval such as $[5, 7]$ from $[3, 7]$ and $[5, 6]$, whose lower end comes from one parent and whose upper end from the other, so the value no longer describes any single path and the critical chain read from it can be wrong. In the margin step, the float $P - t_v$ subtracts two quantities that depend on the same durations. Interval subtraction treats them as independent, which is the dependency problem of Chapter 2. For the dummy start node of a project, for example, P and t_v are the same interval, say $[30, 46]$, and interval subtraction gives $[-16, 16]$ when the true float is exactly 0. The margins were therefore wider than the true range, and negative floats appeared where none exist. The present toolkit computes each interval quantity by the scheme its structure permits, and never by running the kernel on an interval type.

0.6 Exact Interval Floats by the Domination Split

Throughout this section the mode is LongestPath, the durations d_u lie in intervals, the edge values are fixed, and $f_v = P - t_v \geq 0$ is the float of node v . The question is the exact range of f_v over the box of duration configurations.

Proposition 1 (Corner sufficiency). *With all other durations fixed, f_v is a monotone function of any single duration d_u . Consequently the extremes of f_v over the box are attained at corner configurations.*

Proof. A path visits a node at most once, so as a function of $x = d_u$ every path length is either constant or affine with slope one. Hence $P(x) = \max(C_0, C_1 + x)$, where C_1 is the best path through u and C_0 the best avoiding it, and likewise $t_v(x) = \max(A_0, A_1 + x)$ over the paths through v . Each has one breakpoint with slope 0 then 1. If P 's breakpoint comes first their difference has slope pattern $0, +1, 0$, otherwise $0, -1, 0$, and in both cases f_v is monotone in x on the whole line. For the corner claim, push any coordinate to whichever endpoint does not worsen the objective, which monotonicity permits, and repeat per coordinate until the configuration is a corner. $\square$

Lemma 2 (Incomparable coordinates). *If no complete path contains both u and v , then f_v is nondecreasing in d_u , whatever the other durations are.*

Proof. No path through v contains u , so t_v does not depend on d_u , while P is nondecreasing in it. $\square$

Lemma 3 (Dominated coordinates). *If every complete path through u also passes through v , and in particular if $u = v$, then f_v is nonincreasing in d_u , whatever the other durations are.*

Proof. Suppose increasing d_u increases P . The new maximiser must contain u , hence v , hence it is a path through v , so t_v rises to equal P and the float drops to zero. If P does not increase, the float cannot rise because t_v is nondecreasing in every duration. $\square$

The two lemmas cover every duration except those in the *bypass set*

$$H_v = \{u : \text{some complete path contains both } u \text{ and } v, \text{ and some complete path contains } u \text{ but not } v\} \quad (11)$$

the nodes that share a route with v and can also route around it. Figure 2 shows the three classes around one node. In the implementation the classes are found from the ancestor and descendant closures of the graph object together with two traversals: the set of nodes that can reach a sink without passing v , and the set of nodes reachable from a source without passing v . An ancestor of v that cannot reach a sink without v is dominated, as is a descendant that cannot be reached from a source without v ; a node that is neither ancestor nor descendant is incomparable; the rest form H_v .

Theorem 4 (Exactness of the domination split). *The maximum of f_v over the box is attained at a corner with every incomparable duration at its upper bound, every dominated duration at its lower bound, and the bypass durations at some corner of H_v . The minimum is attained dually. Enumerating the $2^{|H_v|}$ bypass corners with the remaining durations pinned therefore yields the exact float range of v .*

Proof. By Proposition 1 an optimum lies at a corner. The three classes partition the durations. At an optimal corner, moving an incomparable duration to its upper bound cannot decrease the maximum by Lemma 2, and moving a dominated duration to its lower bound cannot decrease it by Lemma 3, so some optimal corner has the pinned pattern and lies within the enumerated set. $\square$

Algorithm 3 gives the procedure. Its cost is

$$2 \sum_v 2^{|H_v|} \quad (12)$$

scalar propagations, one maximising and one minimising sweep per node, against 2^k for exhaustive enumeration of all k interval durations. The exponent has moved from the size of the problem to the reconvergent width around each node. One exhaustive sweep serves

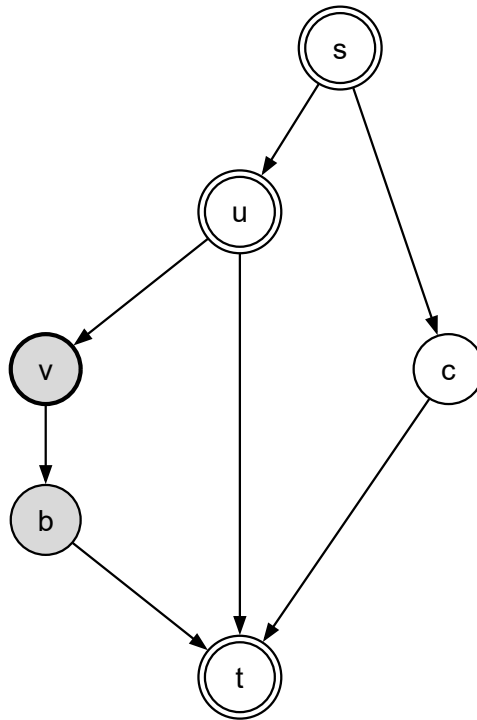


Figure 2: Duration classes relative to node v . Shaded nodes are dominated by v (every complete route through them passes v), the unfilled node is incomparable to it, and double-bordered nodes form the bypass set H_v : they share routes with v and can also route around it.

every node at once, so the implementation computes both counts before running anything and uses whichever is cheaper. It is therefore never worse than exhaustive enumeration, and it refuses when both counts exceed a stated limit. Some instances must remain hard, because the general problem is NP-hard [3], and the split shows where the hardness sits, in networks whose bypass sets are large.

Algorithm 3 Domination split for exact interval floats (LongestPath, point-valued edge values)

Require: graph object, interval durations $[\underline{d}_u, \bar{d}_u]$, scalar edge values, run limit

Ensure: exact float range $[\underline{f}_v, \bar{f}_v]$ for every node v

```

1:  $K \leftarrow$  set of nodes with non-degenerate intervals;  $k \leftarrow |K|$ 
2: for all nodes  $v$  do
3:   classify each  $u \in K$  as incomparable, dominated, or bypass ( $u \in H_v$ )
4: end for
5:  $\text{total} \leftarrow 2 \sum_v 2^{|H_v|}$ 
6: if  $2^k \leq \text{total}$  then run exhaustive enumeration instead and return
7: end if
8: if  $\text{total} > \text{run limit}$  then refuse
9: end if
10: for all nodes  $v$  do
11:   for objective in (maximise, minimise) do
12:     pin incomparable durations at upper (resp. lower) and dominated at lower
        (resp. upper)
13:     for all corners of  $H_v$  do
14:       run Algorithm 1; record  $f_v$ 
15:     end for
16:   end for
17:    $\underline{f}_v \leftarrow$  minimum recorded;  $\bar{f}_v \leftarrow$  maximum recorded
18: end for
19: necessarily critical  $\leftarrow \{v : \bar{f}_v = 0\}$ ; possibly critical  $\leftarrow \{v : \underline{f}_v = 0\}$ 

```

The bypass set is a measure of reconvergence, and this is where the toolkit meets the structural theme of Chapter 4 without using its module. Both analyses pay for routes that divide and meet again, the decomposition module per join through the nodes it must fix, and the split per node through H_v . The two sets index reconvergence at different granularities, neither contains the other in general, and neither computation uses the other.

0.7 Validation

The toolkit was checked against three computations that do not use its kernels. The checks were run on a set of ten networks, listed with their sizes in Appendix B, and they apply to every result reported in Section 0.8.

The first check is a path oracle. It lists every complete path from a source to a sink, evaluates each path under the mode’s operators, and from the list obtains the project value, the best path through each node, each node’s margin and the critical set. On the seven networks small enough to list every path (32 to 306 nodes, 128 to 2,450 paths), the toolkit and the oracle agree to within 3×10^{-14} in every quantity for all four modes, and the critical sets are identical. On the three networks with too many paths to list (2.7×10^6 and more), two weaker checks are used instead: two thousand randomly sampled paths never give a better value than the project value the toolkit reports, and the accumulation total equals the sum of Eq. (7) evaluated from the multiplicities.

The second check is for the interval methods. The path oracle is run at every corner of the input box, which gives the exact margin range directly. The conservative enclosure of Eq. (10) is checked to contain that range, the exhaustive method is checked to equal it, and the domination split is checked to give the same margins and the same necessarily and possibly critical sets as the exhaustive method on every network on which both can run.

The third check is Monte Carlo sampling, used where the exhaustive method is out of reach. Fifty thousand configurations of durations are drawn uniformly from the box, the scalar kernel is run on each, and the resulting floats are checked to lie within the split’s bounds and to reach them.

0.8 Case Studies

Timings in this section are wall-clock measurements on a single core of the second call in a warm process, so that just-in-time compilation is excluded. The convention is stated in Appendix B.

0.8.1 A benchmark project network

The first case study is instance j301_1 of the PSPLIB project scheduling library [8]. PSPLIB instances are activity-on-node networks with a dummy start and a dummy end, each activity carrying a published duration, and they are the standard benchmark set for project scheduling. The instance has 32 nodes, of which 30 are real activities, and 48 precedence edges, with one source, one sink, twelve forks and twelve joins in eleven layers. Its resource requirements are not used: the toolkit computes the precedence-only schedule, which the PSPLIB instance file records as the MPM-Time of the instance. The durations are listed in Appendix A.

Under LongestPath the project completes in 38 units, which matches the MPM-Time of 38 published with the instance, along the chain $1 \rightarrow 3 \rightarrow 8 \rightarrow 12 \rightarrow 14 \rightarrow 17 \rightarrow 22 \rightarrow 23 \rightarrow 24 \rightarrow 30 \rightarrow 32$ (Figure 3). Eleven nodes are critical, nine of them real activities,

and three further activities (4, 10 and 16) carry a slack of one unit (Table 2). Under ShortestPath the fastest complete chain takes 18 units, through activities 2, 4, 6, 10, 25 and 30. The two readings share activity 30 and nothing else, so the chain that bounds the project from above and the chain that bounds it from below run through different parts of the network. Each of the three deterministic passes completes in under 0.2 milliseconds.

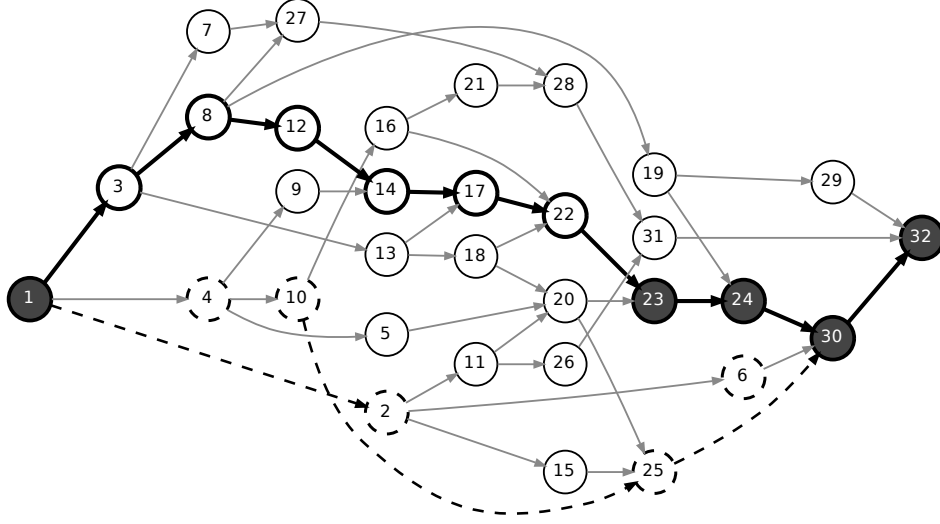


Figure 3: PSPLIB j301_1. Bold nodes and edges form the LongestPath critical chain (completion 38); dashed nodes and edges form the ShortestPath chain (completion 18). The filled nodes are those the interval analysis of Figure 4 finds necessarily critical under ± 20 per cent duration uncertainty.

Read as cost with the same values, the accumulation mode gives a total of 362 at the end node. The activities with the largest contribution are not those with the largest durations. Activity 2, with duration 8, reaches the end node along five routes and contributes 40; activity 3, with duration 4, reaches it along nine routes and contributes 36; activity 16, with duration 10, lies on only two routes and contributes 20 (Table 3). Under a budget of 398.2, ten per cent above the current total, activity 3 can grow by 4.0 units before the budget is reached, activity 16 by 18.1, and activity 6, which lies on a single route, by 36.2. Contribution and allowance order the activities differently.

With every one of the 30 activity durations widened to ± 20 per cent and the two dummies held at zero, the project value lies in $[30.4, 45.6]$. Exhaustive corner enumeration of the floats would require 2^{30} propagations, about 1.1×10^9 . The domination split required 50,524, a factor of about 21,000 fewer, with the largest bypass set holding 13 of the 30 durations and the mean bypass set 6.8. It completed in 4.2 seconds, about 82 microseconds per corner, each corner being one run of Algorithm 1. Five nodes are necessarily critical, the two dummies and activities 23, 24 and 30, so the final three

Table 2: LongestPath on PSPLIB j301_1: the critical chain and the three nearest non-critical activities.

Node	Duration	Completion	Slack
3	4	4	0
8	9	13	0
12	2	15	0
14	3	18	0
17	6	24	0
22	7	31	0
23	2	33	0
24	3	36	0
30	2	38	0
4	6	6	1
10	7	13	1
16	10	23	1

Table 3: Accumulation on PSPLIB j301_1 under a budget of 398.2 at the end node (current total 362). Allowance is headroom divided by path multiplicity.

Node	Duration	Multiplicity	Contribution	Allowance
2	8	5	40	7.2
3	4	9	36	4.0
4	6	6	36	6.0
8	9	4	36	9.1
11	9	3	27	12.1
13	6	4	24	9.1
10	7	3	21	12.1
16	10	2	20	18.1

activities of the deterministic chain are critical in every configuration of the box while the first six are not. Nineteen nodes are possibly critical, and thirteen activities cannot become critical anywhere in the box (Figure 4).

The conservative enclosure of Section 0.5, obtained from the two corner runs alone, certifies no node as necessarily critical and reports 26 nodes as possibly critical. The exact split therefore identifies three necessarily critical activities that the enclosure does not, and removes seven activities from the possibly critical set. The deterministic pass names eleven nodes, the enclosure names 26, and the exact interval analysis names 19 and identifies the three that are certain. Monte Carlo sampling of fifty thousand configurations within the box produced no violation of the bounds. The sampled floats reached the lower and the upper bound at 30 of the 32 nodes, and at the remaining two the bounds are attained by the corner configurations the split itself identified.

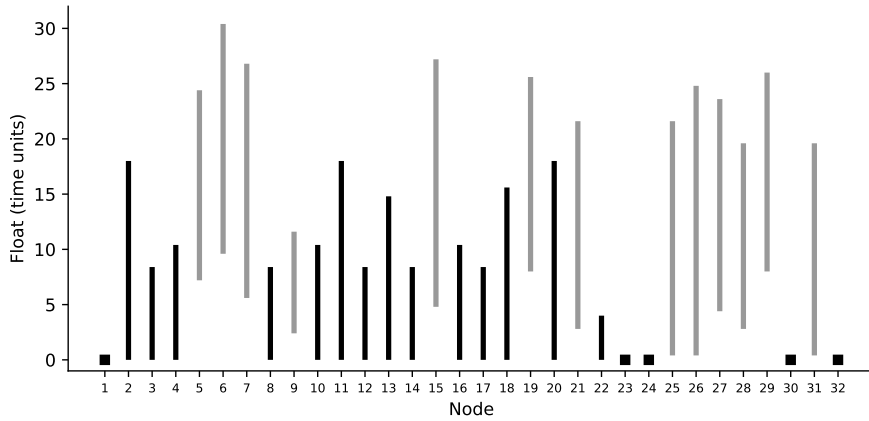


Figure 4: Exact float bounds on PSPLIB j301_1 with every duration at plus or minus 20 per cent. Black bars are the nineteen possibly critical nodes, whose lower bounds reach zero; three real activities and the two dummies have upper bound zero as well and are necessarily critical. Grey bars are the thirteen activities whose floats stay positive everywhere in the box. The five necessarily critical nodes are drawn as points at zero, since both bounds of their float are zero.

0.8.2 The boundary of the split

The advantage of the split disappears under dense mesh reconvergence. On a five-by-five grid DAG with all 25 durations interval-valued, the bypass sets hold up to 23 of the 25 durations, so the split would need 3.9×10^7 propagations against 3.4×10^7 for one shared exhaustive sweep. The never-worse rule therefore selects the exhaustive sweep, and the instance is at the boundary of the method. The hardness results guarantee that some regime behaves this way, and it is the same regime in which the conditioning sets of the probability toolkit grow.

0.9 Discussion

The domination split is proved and implemented for the LongestPath mode only. The ShortestPath dual mirrors Section 0.6 but has not been derived in full, and the toolkit refuses the split for any other mode. For those modes the exact interval margins are available only by exhaustive corner enumeration.

The split requires point-valued edge values. An interval-valued edge reduces to an interval duration by subdividing the edge with a node that carries the value, so the reduction is known, but the toolkit does not yet perform it and refuses interval edge values on the split path.

Probability-box durations are not supported. The forward quantities would propagate by the p-box arithmetic of Chapter 2, but the margins would meet the same dependence problem as the intervals, and no corner argument applies to them. Therefore no method for them is claimed.

The cost of the split grows with the bypass sets, and on dense meshes the sets approach the whole network. The never-worse rule keeps the cost at or below that of exhaustive enumeration, and no lower.

The boundary of the toolkit is set by the algebra. A new quantity is added by giving its operator pair, and the pair obtains a backward pass in the same way as the four provided modes, by carrying a residual or by being linear. Whether a proposed pair has this property is a mathematical question that the implementation does not check, and showing it is the responsibility of whoever adds the mode. Two extensions follow the existing arguments: neighbouring nodes have overlapping bypass sets and therefore overlapping corner evaluations, which could be shared in the way the hash-consed store of Chapter 4 shares diamonds, and the edge subdivision above would admit interval edge values. These are listed in Chapter 11.

0.10 Summary

In this chapter the Critical Path Toolkit has been presented. The classical critical path method has been generalised to a family of analysis modes, each an operator pair with the backward semantics its algebra supports, and four modes have been provided: LongestPath, ShortestPath and MaxScaling on one order-based kernel, and Accumulation on a linear kernel whose backward object is the adjoint. Under interval durations the forward quantities have been computed exactly from two scalar runs, and the floats exactly by the domination split, whose correctness rests on three short proofs and whose cost grows with the reconvergent width around each node instead of with the number of uncertain inputs. The toolkit has been validated against a path oracle, exhaustive corner enumeration and Monte Carlo sampling, and applied to a benchmark project network. The MaxScaling

mode is demonstrated on the integrated case study of Chapter ?? . On a dense mesh the split reaches its boundary and the toolkit falls back to exhaustive enumeration, as the hardness of the general problem requires.

Bibliography

- [1] J. E. Kelley and M. R. Walker, “Critical-path planning and scheduling,” in *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference*, Boston, MA, 1959, pp. 160–173. DOI: 10.1145/1460299.1460318
- [2] D. G. Malcolm, J. H. Roseboom, C. E. Clark and W. Fazar, “Application of a technique for research and development program evaluation,” *Operations Research*, vol. 7, no. 5, pp. 646–669, 1959. DOI: 10.1287/opre.7.5.646
- [3] S. Chanas and P. Zieliński, “The computational complexity of the criticality problems in a network with interval activity times,” *European Journal of Operational Research*, vol. 136, no. 3, pp. 541–550, 2002. DOI: 10.1016/S0377-2217(01)00048-0
- [4] S. Chanas and P. Zieliński, “On the hardness of evaluating criticality of activities in a planar network with duration intervals,” *Operations Research Letters*, vol. 31, no. 1, pp. 53–59, 2003.
- [5] D. Dubois, H. Fargier and J. Fortin, “Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks,” *Journal of Intelligent Manufacturing*, vol. 16, pp. 407–421, 2005. DOI: 10.1007/s10845-005-1654-5
- [6] J. Fortin, P. Zieliński, D. Dubois and H. Fargier, “Criticality analysis of activity networks under interval uncertainty,” *Journal of Scheduling*, vol. 13, pp. 609–627, 2010. DOI: 10.1007/s10951-010-0163-3
- [7] F. Baccelli, G. Cohen, G. J. Olsder and J.-P. Quadrat, *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Chichester: John Wiley & Sons, 1992.
- [8] R. Kolisch and A. Sprecher, “PSPLIB – a project scheduling problem library,” *European Journal of Operational Research*, vol. 96, no. 1, pp. 205–216, 1997. DOI: 10.1016/S0377-2217(96)00170-1